

ROYMS

This initial phase concentrated on defining the project's functional base and starting the design work. The effort dedicated to this core analysis was approximately 2 hours and 30 minutes. This work began with 30 minutes spent on selecting the main subject area and formally choosing the project's name. Following this, we spent a significant block of 2 hours refining the planned 10 use cases by adding specific, real-world functionalities to each. We then required an additional 1 hour and 30 minutes to finalize these use cases and ensure they all followed the same template, ensuring consistency. To meet our schedule and stay on track, we had to work remotely, and I set a clear deadline of November 20th for the completion of this definition work.

On November 21st, we started with a review of the completed use cases. This was immediately followed by the creation of the UML diagrams for each use case, a process which took us around 45 minutes. We chose to use PlantUML specifically to maintain a consistent style across all diagrams. Next, the focus shifted to designing and creating the Mockups, which took approximately 1 hour. We decided on a standard template and initially used Draw.io for the design work. However, this tool was not ideal due to its poor website usability. Each team member was responsible for creating the mockups related to the use cases they had worked on. I personally delegated most of the mockup creation and only completed one (the notification mockup), as I needed to allocate 30 to 45 minutes to start drafting this Management Report.

Despite the intense workload, we faced no major problems that we couldn't resolve within the team. A significant positive factor has been the strong team dynamic; we have a good team, which has created a positive working relationship. This excellent collaboration has made task distribution extremely easy and helped us solve potential scheduling or allocation issues quickly before they could escalate. Overall, I am very satisfied with the team's productivity and efficiency during this first period of focused investment.

Use Cases Description

1. Game Catalog Management.....	7
1.1. Brief Description.....	7
1.2. Flow of Events.....	7
1.2.1. Basic Flow.....	7
1.2.1.1. Add a Game.....	7
1.2.1.2. Edit a game.....	7
1.2.1.3. Delete a game.....	8
1.2.2. Alternative Flows.....	8
1.2.2.1. Game Not Found.....	8
1.2.2.2. Delete Cancelled.....	8
1.3. Special Requirement.....	8
1.4. Pre-Conditions.....	8
1.5. Post-Conditions.....	9
1.6. Extension Points.....	9
2. Player Management.....	10
2.1. Brief Description.....	10
2.2. Flow of Events.....	10
2.2.1. Basic Flow.....	10
2.2.2. Alternative Flow.....	10
2.2.2.1. Profile Modification by Owner.....	10
2.2.2.2. Attempted Modification Without Rights.....	11
2.2.2.3. Missing or Invalid Information During Modification.....	11
2.3. Special Requirement.....	11
2.4. Pre-Conditions.....	11
2.5. Post-Conditions.....	11
2.6. Extension Points.....	11
3. Team Management.....	12
3.1. Brief Description.....	12
3.2. Flow of Events.....	12
3.2.1. Basic Flow.....	12
3.2.1.1. Team Management Sub-Flow (Invite/View Roster).....	12
3.2.1.2. Invite a Player Sub-Flow.....	13
3.2.1.3. Handle Join Request Sub-Flow.....	13
3.2.2. Alternative Flow.....	13
3.2.2.1. Team Name Already Taken.....	13
3.2.2.2. Player Joins a Team (Non-Captain Action).....	13
3.2.2.3. Player Leaves a Team (Transfer/Dissolve).....	14

3.3. Special Requirement.....	14
3.4. Pre-Conditions.....	14
3.5. Post-Conditions.....	14
3.6. Extension Points.....	14
4. Match Management.....	15
4.1. Brief Description.....	15
4.2. Flow of Events.....	15
4.2.1. Basic Flow.....	15
4.2.2. Alternative Flow.....	16
4.2.2.1. Match Not Available for Detailing.....	16
4.2.2.2. Schedule Conflict Detected.....	16
4.2.2.3. Composition or Personnel Validation Failure.....	16
4.3. Special Requirement.....	16
4.4. Pre-Conditions.....	16
4.5. Post-Conditions.....	16
4.6. Extension Points.....	17
5. Basket Management.....	18
5.1. Brief Description.....	18
5.2. Flow of Events.....	18
5.2.1. Basic Flow.....	18
5.2.1.1. Add Item to Basket.....	18
5.2.1.2. Update Item Quantity.....	18
5.2.1.3. Remove Item from Basket.....	18
5.2.1.4. Empty Basket.....	18
5.2.2. Alternative Flows.....	18
5.2.2.1. Item Out of Stock.....	18
5.2.2.2. Invalid Quantity.....	18
5.2.2.3. Basket Already Empty.....	19
5.3. Special Requirements.....	19
5.4. Pre-Conditions.....	19
5.5. Post-Conditions.....	19
5.6. Extension Points.....	19
6. Tournament Management.....	20
6.2. Flow of Events.....	20
6.2.1. Basic Flow This use case begins when an Organizer wishes to perform administrative tasks on a tournament.....	20
6.2.1.1. Create a Tournament.....	20
6.2.1.2. Manage Registrations.....	20
6.2.1.3. Generate Bracket.....	21
6.2.1.4. Submit and Validate a Result.....	21
6.2.1.5. Modify or Cancel Tournament.....	21
6.2.2. Alternative Flows.....	21
6.2.2.1. Invalid Configuration Data.....	21
6.2.2.2. Insufficient Participants for Bracket.....	21

6.2.2.3. Result Dispute.....	22
6.3. Special Requirements.....	22
6.4. Pre-Conditions.....	22
6.5. Post-Conditions.....	22
6.2.1.1. Create a Tournament.....	22
6.2.1.2. Manage Registrations.....	23
6.2.1.3. Generate Bracket.....	23
6.2.1.4. Submit and Validate a Result.....	23
6.2.1.5. Modify or Cancel Tournament.....	23
6.2.2. Alternative Flows.....	24
6.2.2.1. Invalid Configuration Data.....	24
6.4. Pre-Conditions.....	24
6.5. Post-Conditions.....	24
7. Notification System.....	25
7.1 Brief Description.....	25
7.2 Flow of Events.....	25
• 7.2.1 Basic Flow.....	25
○ 7.2.2 Sub-Flows by Domain.....	25
■ 7.2.2.1 Game Catalog Notifications.....	25
■ 7.2.2.2 Player Profile Notifications.....	25
■ 7.2.2.3 Team Management Notifications.....	25
■ 7.2.2.4 Match Management Notifications.....	25
■ 7.2.2.5 Basket Notifications.....	25
■ 7.2.2.6 Tournament Notifications.....	25
○ 7.2.3 Alternative Flows.....	26
■ 7.2.3.1 Notification Delivery Failure.....	26
■ 7.2.3.2 User Opt-Out.....	26
■ 7.2.3.3 Invalid Trigger Data.....	26
■ 7.2.3.4 Notification Read/Cleared.....	26
■ 7.2.3.5 Notification Expiry.....	26
7.3 Special Requirements.....	26
7.4 Pre-Conditions.....	26
7.5 Post-Conditions.....	26
7.6 Extension Points.....	26
8. Payment Management.....	27
8.1 Brief Description.....	27
8.2 Flow of Events.....	27
8.2.1 Basic Flow.....	27
8.2.1.1 Apply Promo Code.....	27
8.2.1.2 Confirm Payment.....	27
8.3 Alternative Flows.....	27
8.3.1 Invalid Promo Code.....	27
8.3.2 Payment Declined.....	27
8.3.3 Session Timeout.....	27

8.4 Special Requirements.....	28
8.5 Pre-Conditions.....	28
8.6 Post-Conditions.....	28
8.7 Extension Points.....	28
9. Product Management.....	29
9.1 Brief Description.....	29
9.2 Flow of Events.....	29
9.2.1 Basic Flow.....	29
9.2.1.1 Add Product.....	29
9.2.1.2 Edit Product.....	29
9.2.1.3 Delete Product.....	29
9.2.1.4 Categorize Product.....	29
9.3 Alternative Flows.....	29
9.3.1 Product Not Found.....	30
9.3.2 Invalid Input.....	30
9.3.3 Product Linked to Active Orders.....	30
9.4 Special Requirements.....	30
9.5 Pre-Conditions.....	30
9.6 Post-Conditions.....	30
9.7 Extension Points.....	30
10. Maintain User Management.....	31
10.1 Brief Description.....	31
10.2 Flow of Events.....	31
10.2.1 Basic Flow.....	31
10.2.1.1 Create Account.....	31
10.2.1.2 Login.....	32
10.2.1.4 Assign Role.....	32
10.2.1.5 View/Edit Profile.....	32
10.2.1.7 Password Reset / Forgot Password.....	32
10.2.1.8 Deactivate/Reactivate Account.....	32
10.3 Alternative Flows.....	32
10.3.1 Invalid Credentials.....	33
10.3.2 Duplicate Account.....	33
10.3.3 Unauthorized Role Assignment.....	33
10.3.4 Invalid Password Reset Request.....	33
10.3.6 Deactivation Cancelled.....	33
10.4 Special Requirements.....	33
10.5 Pre-Conditions.....	33
10.6 Post-Conditions.....	33
10.7 Extension Points.....	33

1. Game Catalog Management

1.1. Brief Description

This use case allows the administrator to maintain games available for tournaments. This includes adding, editing or deleting a Game

1.2. Flow of Events

1.2.1. *Basic Flow*

- 1) The system requests that the administrator specify the function he/she would like to perform (either Add a Game, Update a Game, or Delete a Game)
- 2) Once the administrator provides the requested information, one of the sub flows is executed.
If the administrator selected “Add a Game”, the **Add a Game** subflow is executed.
If the administrator selected “Edit a Game”, the **Edit a Game** subflow is executed.
If the administrator selected “Delete a Game”, the **Delete a Game** subflow is executed.

1.2.1.1. Add a Game

The system requests that the administrator enter the game information. This includes:

- Name of the game
- Editor of the game
- Date of Release
- Banner

Once the administrator provides all the information and submits, the system generates and assign a unique game id number to the game and the game is added to the system

1.2.1.2. Edit a game

- 1) The systems requests that the administrator selects the game
- 2) Once the administrator provides the game, the system retrieves and displays the game information.

- 3) The administrator then makes the desired changes on the information specified on the Add a game subflow.
- 4) Once the administrator submits, the system updates the game record with the updated information.

1.2.1.3. Delete a game

- 1) The system requests that the administrator specify the game.
- 2) Once the administrator provides the game id, the system displays ask the administrator to confirm the deletion of the game
- 3) The administrator confirms the deletions
- 4) The system removes the game from the system, as well as any information related to the game.

1.2.2. *Alternative Flows*

1.2.2.1. Game Not Found

If in the **Edit a Game or Delete a Game** sub-flows, a game does not exist, the system displays an error message. The administrator can then enter a different id number or cancel the operation, at which point the use case ends.

1.2.2.2. Delete Cancelled

If in the **Delete a Game sub -flow**, the administrator decides not to confirm the deletion, the delete is cancelled and the **Basic Flow** is re-started at the beginning.

1.3. Special Requirement

None.

1.4. Pre-Conditions

The administrator must be logged onto the system before this use case begins.

1.5. Post-Conditions

If the use case was successful, the game information is added, updated, or deleted from the system. Otherwise, the system state is unchanged.

1.6. Extension Points

None.

2. Player Management

2.1. Brief Description

This use case allows any **User** (logged in or not) to view a player's public information. It also allows a **Player** to modify their own profile data (e.g., username, avatar, biography) if, and only if, the profile being viewed belongs to the logged-in user.

2.2. Flow of Events

2.2.1. *Basic Flow*

This use case begins when a User wishes to view a player's profile.

1. The User selects or searches for a player whose profile they wish to view.
2. The system retrieves and displays the public information of the selected player's profile. This includes:
 - Username and Avatar
 - Biography
 - Statistics and performance records.

2.2.2. *Alternative Flow*

2.2.2.1. Profile Modification by Owner

- If in the Basic Flow (Step 2), the User is logged in and the User's identity matches the identity of the player whose profile is displayed, the User can **Edit Profile**.
- The User selects the Edit Profile option.
- The system displays the **private** and **public** profile information in editable fields.
- The User makes the desired changes (username, avatar, biography, etc.), then select **Save**.
- The system validates the entered data (format, forbidden characters, username uniqueness).

- The system updates the Player's profile record.
- The system confirms to the Player that their profile has been successfully updated. The use case ends.

2.2.2.2. Attempted Modification Without Rights

- If, in the Basic Flow (Step 2), the User is logged in but their identity does not match the identity of the player whose profile is displayed, the Edit Profile option is not shown.
- The use case ends after consultation

2.2.2.3. Missing or Invalid Information During Modification

- If, during the *Profile Modification by Owner* sub-flow, a validation fails (e.g., username already taken, data is improperly formatted), the system displays an error message and prompts the Player to correct the information.
- The Player may choose to correct the information or cancel the operation, in which case the use case ends without modification.

2.3. Special Requirement

The updating of statistics and performance records must be performed automatically by the system after the validation of each match result (see: **Submit and Validate a Result** use case).

2.4. Pre-Conditions

- Consultation : No pre-conditions required to view public information
- Modification: The player must be logged into the system

2.5. Post-Conditions

- If the modification was successful, the Player's profile information is updated in the system.
- Otherwise, the system state remains unchanged

2.6. Extension Points

None.

3. Team Management

3.1. Brief Description

This use case allows a Player to create a team, manage its roster (invite players, accept/reject requests), and handle the disbanding or transfer of captaincy when a player leaves the team.

3.2. Flow of Events

3.2.1. Basic Flow

This use case begins when a Player wishes to create a new team.

1. The Player requests to create a new team.
 2. The system requests the Player to enter the team details, including:
 - Team Name
 - Team Tag (short acronym)
 - Team Avatar/Logo
 3. The Player provides the requested information.
 4. The system validates the information (Team Name uniqueness, format rules).
 5. The system creates the team and generates a unique Team ID.
 6. The Player is automatically assigned the role of **Captain**.
 7. The system confirms the creation and displays the new team's profile. The use case continues to the **Team Management Sub-Flow**.
- 3.2.1.1. Team Management Sub-Flow (Invite/View Roster)
- The system displays the current team roster and available management options.
 - The Captain may choose one of the following actions:
 - **Invite a Player** (executes the *Invite a Player Sub-Flow*).
 - **View/Change Team Details** (updates name, tag, or avatar).
 - **Accept/Reject Join Requests** (executes the *Handle Join Request Sub-Flow*).

3.2.1.2. Invite a Player Sub-Flow

1. The Captain enters the username(s) or ID(s) of the player(s) to invite.
2. The system verifies that the invited players are registered and not already on a team.
3. The system sends an invitation notification to the selected player(s).
4. The invitation status is set to "Pending" in the system. The use case returns to the Team Management Sub-Flow.

3.2.1.3. Handle Join Request Sub-Flow

1. The system displays a list of pending requests from players who wish to join the team (see Alternative Flow: *Player Joins a Team*).
2. The Captain selects a request and chooses to Accept or Reject it.
3. If Accepted, the system adds the Player to the team roster and sends a confirmation notification.
4. If Rejected, the system removes the request and sends a rejection notification.
5. The use case returns to the Team Management Sub-Flow.

3.2.2. *Alternative Flow*

3.2.2.1. Team Name Already Taken

- If, in the Basic Flow (Step 4), the team name or tag is not unique, the system displays an error message.
- The Player can choose to enter a different name/tag or cancel the operation, at which point the use case ends.

3.2.2.2. Player Joins a Team (Non-Captain Action)

- A non-Captain Player initiates this action by finding the team (via a separate search function) and requesting to join.
- The system verifies the Player is not already on a team and sends a notification to the Captain.
- The request status is set to "Pending." The use case ends, pending the Captain's approval (handled in the *Handle Join Request Sub-Flow*).

3.2.2.3. Player Leaves a Team (Transfer/Dissolve)

- A Player (Captain or Member) requests to leave the team.
- The system verifies the Player's role:
 - If the Player is a **Member**: The system removes the Player from the roster and sends a notification to the Captain. The use case ends.
 - If the Player is the **Captain**: The system prompts the Captain to either:
 1. **Transfer Captaincy** to an existing member.
 2. **Dissolve the Team** (if no other members, or if requested).
- If Captaincy is transferred, the system updates the roles and sends a notification. The old Captain is then removed from the roster. The use case ends.
- If the team is dissolved, the system marks the team record for deletion and notifies all members. The use case ends.

3.3. **Special Requirement**

Team name and tag validation must be valid

3.4. **Pre-Conditions**

The player must be logged into the system

3.5. **Post-Conditions**

- If a team was created, a new team record exists, and the Player is the Captain.
- If an invitation was accepted or a request handled, the team roster is updated.
- If the team was dissolved, the team record is marked for deletion.
- Otherwise, the system state remains unchanged.

3.6. **Extension Points**

None.

4. Match Management

4.1. Brief Description

This use case allows the Organizer to input, update, and finalize all details for a match that has been generated in the tournament bracket. This includes setting the Date and Time, assigning Referee(s), assigning or verifying Coach(es) for each team. The coach can specify the definitive Team Composition for the match.

4.2. Flow of Events

4.2.1. *Basic Flow*

This use case begins when the **Organizer** needs to specify all the necessary details for a scheduled match.

The Organizer selects a specific match from the tournament bracket that requires detailing.

The system retrieves and displays the current match record, including the participating teams.

The system requests the Organizer to input or verify the following details for the match:

- Date and Time of the Match
- Referee(s) Assignment
- Coach Assignment: The Organizer selects or confirms the **Coach** for each participating team from the registered personnel.
- Team Composition: The Coach selects the specific players from each team's full roster to form the definitive **Composition** for the match, adhering to the game's limits.

The Organizer enters or updates all requested information, then selects Save.

The system validates all input: checking for scheduling conflicts (referee availability), ensuring players are on the correct roster, and verifying the composition meets match rules (player count).

The system updates the match record with all new logistical, personnel, and composition details.

The system confirms the successful update and automatically sends notifications to the participating teams, assigned referees, and coaches regarding the

finalized match details (schedule, composition, assigned officials). The use case ends.

4.2.2. *Alternative Flow*

4.2.2.1. Match Not Available for Detailing

If, in the Basic Flow (Step 1), the selected match does not exist or is not available for detailing (e.g., already completed), the system displays an error message.

The Organizer acknowledges the message, and the use case ends.

4.2.2.2. Schedule Conflict Detected

If, during validation (Step 6), the new match time conflicts with the availability of an assigned Referee or if a team is double-booked, the system displays a warning about the conflict.

The Organizer must correct the date/time or reassign the referee.

4.2.2.3. Composition or Personnel Validation Failure

If, during validation (Step 6), the system detects an issue (e.g., an assigned coach is suspended, or a composition contains too few players), the system displays an error message detailing the issue.

The Organizer must correct the invalid data or cancel the operation, in which case the match record remains unchanged.

4.3. **Special Requirement**

None.

4.4. **Pre-Conditions**

The Organizer must be logged into the system.

The match must have already been generated in the tournament bracket.

4.5. **Post-Conditions**

The selected match record is comprehensively updated with the finalized date, time, assigned referees, coaches, and the definitive team compositions.

Notifications are sent to all affected parties.

Otherwise, the system state remains unchanged.

4.6. Extension Points

None.

5. Basket Management

5.1. Brief Description

This use case allows a user to manage their shopping basket in the RYMS app. It includes viewing offers, adding/removing items, updating quantities, and emptying the basket.

5.2. Flow of Events

5.2.1. *Basic Flow*

This use case starts when the user wishes to manage their basket.

- The system displays available offers (tickets and merchandise).
- The user selects an item to add to the basket.
- The system adds the item to the basket and confirms.
- The user may view the basket contents.
- The user may update the quantity of an item.
- The user may remove an item or empty the basket entirely.

5.2.1.1. Add Item to Basket

- The system displays item details.
- The user selects quantity and clicks “Add to Basket.”
- The system validates availability and adds the item.

5.2.1.2. Update Item Quantity

- The user selects an item in the basket.
- The user changes the quantity.
- The system validates and updates the basket.

5.2.1.3. Remove Item from Basket

- The user selects an item to remove.
- The system removes the item and updates the basket.

5.2.1.4. Empty Basket

- The user selects “Empty Basket.”
- The system prompts for confirmation.
- Upon confirmation, the system clears all items.

5.2.2. *Alternative Flows*

5.2.2.1. Item Out of Stock

If the selected item is unavailable, the system displays an error and prevents addition.

5.2.2.2. Invalid Quantity

If the user enters a quantity exceeding available stock, the system prompts for correction.

5.2.2.3. Basket Already Empty

If the user tries to empty an already empty basket, the system displays a message.

5.3. Special Requirements

- Real-time stock validation
- Responsive basket updates
- Confirmation prompts for destructive actions

5.4. Pre-Conditions

- User is logged in or browsing as guest
- Offers are available

5.5. Post-Conditions

- Basket is updated with the latest user actions
- Items are added, removed, or modified

5.6. Extension Points

- Save basket for later
- Share basket with another user

6. Tournament Management

6.1. Brief Description

This use case allows the Organizer to manage the lifecycle of an e-sport tournament. This includes creating and configuring a new tournament, managing participant registrations (validating or rejecting), generating the tournament bracket, validating match results to advance the tournament, and modifying or canceling the event if necessary.

6.2. Flow of Events

6.2.1. Basic Flow This use case begins when an Organizer wishes to perform administrative tasks on a tournament.

1. The system requests the Organizer to select a specific function: Create a Tournament, Manage Registrations, Generate Bracket, Validate Results, or Modify/Cancel Tournament.
2. Once the Organizer selects an option, the corresponding sub-flow is executed.

6.2.1.1. Create a Tournament

1. The Organizer requests to create a new tournament.
2. The system prompts for tournament configuration details (e.g., Name, Game selection, Start Date, Format, Max number of participants).
3. The Organizer enters the configuration details and chooses to "Publish."
4. The system validates the input data.
5. The system creates the tournament record and opens it for public viewing or registration.

6.2.1.2. Manage Registrations

1. The Organizer selects a tournament with pending registrations.
2. The system displays a list of players or teams who have requested to join.
3. The Organizer reviews the applicants and selects "Validate" or "Reject" for specific requests.
4. The system updates the registration status:
 - If Validated: The participant is added to the official tournament roster.
 - If Rejected: The participant is removed from the queue, and a notification is sent.

6.2.1.3. Generate Bracket

1. The Organizer selects a tournament where registrations are closed.
2. The Organizer requests to generate the bracket.
3. The system checks if the required number of participants is met.
4. The system automatically creates the matches and the bracket structure.
5. The system notifies participants that the bracket is live.

6.2.1.4. Submit and Validate a Result

1. The Organizer views a match where players have submitted scores.
2. The system displays the reported scores and any potential disputes.
3. The Organizer confirms the correct score or corrects erroneous data.
4. The system validates the result, updates the bracket, and advances the winner to the next round.
5. The system updates player statistics automatically.

6.2.1.5. Modify or Cancel Tournament

1. The Organizer selects an active tournament.
2. The Organizer updates specific parameters (e.g., dates, rules) or selects "Cancel Tournament."
3. If updating: The system saves changes and notifies participants.
4. If canceling: The system updates the status to "Cancelled" and notifies all registered participants.

6.2.2. Alternative Flows

6.2.2.1. Invalid Configuration Data

If during the "Create a Tournament" or "Modify Tournament" sub-flows, the Organizer enters invalid data (e.g., a start date in the past, missing required fields), the system displays an error message. The Organizer must correct the data or cancel the operation.

6.2.2.2. Insufficient Participants for Bracket

If the Organizer attempts to "Generate Bracket" (Sub-flow 5.2.1.3) but the number of validated participants is insufficient or does not match the tournament format rules, the system displays an error. The Organizer must wait for more registrations or modify the tournament format.

6.2.2.3. Result Dispute

If a player contests a result before validation, the system flags the match for manual review. The Organizer must investigate (consulting proofs/logs) before performing the "Validate a Result" step (Sub-flow 5.2.1.4).

6.3. Special Requirements

- **Bracket Automation:** The system must be capable of automatically generating match pairings based on standard tournament rules (e.g., single elimination) when triggered.
- **Notifications:** The system must send automated notifications for registration status changes and bracket updates.

6.4. Pre-Conditions

- The Organizer must be logged into the system.
- For flows involving existing tournaments, a tournament must have been previously created.

6.5. Post-Conditions

- **Create:** A new tournament is available in the catalog.
- **Registrations:** The roster is updated, and participants are notified.
- **Bracket:** The match structure is generated and visible to users.
- **Results:** The tournament progresses, and player statistics are updated.
- **Cancel:** The tournament is no longer active, and participants are refunded/notified (if applicable).

5. Tournament Management

5.1. Brief Description

This use case allows the Organizer to manage the lifecycle of an e-sport tournament. This includes creating and configuring a new tournament, managing participant registrations (validating or rejecting), generating the tournament bracket, validating match results to advance the tournament, and modifying or canceling the event if necessary.

5.2. Flow of Events

5.2.1. Basic Flow

This use case begins when an Organizer wishes to perform administrative tasks on a tournament.

1. The system requests the Organizer to select a specific function: Create a Tournament, Manage Registrations, Generate Bracket, Validate Results, or Modify/Cancel Tournament.
2. Once the Organizer selects an option, the corresponding sub-flow is executed.

6.2.1.1. Create a Tournament

1. The Organizer requests to create a new tournament.

2. The system prompts for tournament configuration details (e.g., Name, Game selection, Start Date, Format, Max number of participants).
3. The Organizer enters the configuration details and chooses to "Publish."
4. The system validates the input data.
5. The system creates the tournament record and opens it for public viewing or registration.

6.2.1.2. Manage Registrations

1. The Organizer selects a tournament with pending registrations.
2. The system displays a list of players or teams who have requested to join.
3. The Organizer reviews the applicants and selects "Validate" or "Reject" for specific requests.
4. The system updates the registration status:
 - o If Validated: The participant is added to the official tournament roster.
 - o If Rejected: The participant is removed from the queue, and a notification is sent.

6.2.1.3. Generate Bracket

1. The Organizer selects a tournament where registrations are closed.
2. The Organizer requests to generate the bracket.
3. The system checks if the required number of participants is met.
4. The system automatically creates the matches and the bracket structure.
5. The system notifies participants that the bracket is live.

6.2.1.4. Submit and Validate a Result

1. The Organizer views a match where players have submitted scores.
2. The system displays the reported scores and any potential disputes.
3. The Organizer confirms the correct score or corrects erroneous data.
4. The system validates the result, updates the bracket, and advances the winner to the next round.
5. The system updates player statistics automatically.

6.2.1.5. Modify or Cancel Tournament

1. The Organizer selects an active tournament.
2. The Organizer updates specific parameters (e.g., dates, rules) or selects "Cancel Tournament."

3. If updating: The system saves changes and notifies participants.
4. If canceling: The system updates the status to "Cancelled" and notifies all registered participants.

6.2.2. Alternative Flows

6.2.2.1. Invalid Configuration Data

If during the "Create a Tournament" or "Modify Tournament" sub-flows, the Organizer enters invalid data (e.g., a start date in the past, missing required fields), the system displays an error message. The Organizer must correct the data or cancel the operation.

5.2.2.2. Insufficient Participants for Bracket

If the Organizer attempts to "Generate Bracket" (Sub-flow 5.2.1.3) but the number of validated participants is insufficient or does not match the tournament format rules, the system displays an error. The Organizer must wait for more registrations or modify the tournament format.

5.2.2.3. Result Dispute

If a player contests a result before validation, the system flags the match for manual review. The Organizer must investigate (consulting proofs/logs) before performing the "Validate a Result" step (Sub-flow 5.2.1.4).

5.3. Special Requirements

- **Bracket Automation:** The system must be capable of automatically generating match pairings based on standard tournament rules (e.g., single elimination) when triggered.
- **Notifications:** The system must send automated notifications for registration status changes and bracket updates.

6.4. Pre-Conditions

- The Organizer must be logged into the system.
- For flows involving existing tournaments, a tournament must have been previously created.

6.5. Post-Conditions

- **Create:** A new tournament is available in the catalog.
- **Registrations:** The roster is updated, and participants are notified.
- **Bracket:** The match structure is generated and visible to users.
- **Results:** The tournament progresses, and player statistics are updated.
- **Cancel:** The tournament is no longer active, and participants are refunded/notified (if applicable).

7. Notification System

7.1 Brief Description

The Notification System ensures timely and relevant alerts are sent to users based on actions and events occurring across the application. It supports system-wide communication for confirmations, warnings, errors, and reminders.

7.2 Flow of Events

- *7.2.1 Basic Flow*
 - A triggering event occurs in one of the application modules (e.g., a match is scheduled, a player is invited).
 - The system identifies the relevant recipients.
 - A notification is generated and dispatched via the selected channel (in-app, email, push).
 - The user receives and views the notification.
 - *7.2.2 Sub-Flows by Domain*
 - *7.2.2.1 Game Catalog Notifications*
 1. Trigger: A game is added, edited, or deleted.
 2. Recipients: Administrators, subscribed players.
 3. Notification Types: Update alerts, deletion warnings.
 - *7.2.2.2 Player Profile Notifications*
 1. Trigger: Profile updated or unauthorized modification attempted.
 2. Recipients: Player, admin (if security breach).
 3. Notification Types: Confirmation, security alert, error message.
 - *7.2.2.3 Team Management Notifications*
 1. Trigger: Team invites, join requests, roster changes, team dissolution.
 2. Recipients: Invited players, captains, team members.
 3. Notification Types: Invitation received, join request accepted/rejected, team update alerts.
 - *7.2.2.4 Match Management Notifications*
 1. Trigger: Match scheduled, updated, or result validated.
 2. Recipients: Participating teams and players.
 3. Notification Types: Match reminder, result confirmation, conflict alert.
 - *7.2.2.5 Basket Notifications*
 1. Trigger: Item added/removed, stock issues, idle basket.
 2. Recipients: Player.
 3. Notification Types: Confirmation, stock alert, reminder.
 - *7.2.2.6 Tournament Notifications*
 1. Trigger: Tournament created, registration confirmed, bracket generated, result disputed.
 2. Recipients: Organizers, registered teams/players.
 3. Notification Types: Registration status, bracket updates, dispute

alerts.

- 7.2.3 *Alternative Flows*

- 7.2.3.1 Notification Delivery Failure
 1. System fails to deliver notification due to connectivity or configuration issues.
 2. Retry mechanism or fallback channel activated.
- 7.2.3.2 User Opt-Out
 1. User has disabled notifications for certain events.
 2. System suppresses notification.
- 7.2.3.3 Invalid Trigger Data
 1. Event data is incomplete or malformed.
 2. Notification generation aborted, error logged.
- 7.2.3.4 Notification Read/Cleared
 1. After viewing a notification, the actor may mark it as "Read" or clear it from the list.
 2. The system updates the notification status in the database.
- 7.2.3.5 Notification Expiry
 1. For time-sensitive notifications (e.g., match invitations), if the specified response time is exceeded, the system automatically marks the notification as expired and removes it from the 'active' list.

7.3 Special Requirements

- Role-based visibility and permissions.
- Real-time delivery for time-sensitive events.
- Multi-channel support (in-app, email, push).
- Localization and language preferences.

7.4 Pre-Conditions

- The triggering event must be successfully completed.
- The recipient must exist and be eligible to receive notifications.

7.5 Post-Conditions

- The notification is logged and visible to the user.
- The user may act on the notification (e.g., accept invite, view match details).

7.6 Extension Points

- Integration with external messaging services (e.g., Firebase, Twilio).
- Notification preferences panel for users.
- Analytics dashboard for notification engagement.

8. Payment Management

8.1 Brief Description

This use case allows a user to apply promo codes and complete payment for items in the basket.

8.2 Flow of Events

8.2.1 Basic Flow

This use case starts when the user proceeds to checkout.

1. The system displays the payment summary.
2. The user enters a promo code (optional).
3. The system validates and applies the discount.
4. The user selects a payment method.
5. The user enters payment details.
6. The system processes the payment.
7. The system displays confirmation and receipt.

8.2.1.1 Apply Promo Code

1. The user enters a promo code.
2. The system checks validity and applicability.
3. If valid, the discount is applied.

8.2.1.2 Confirm Payment

1. The user reviews the final amount.
2. The user confirms the payment.
3. The system processes the transaction.

8.3 Alternative Flows

8.3.1 Invalid Promo Code

If the promo code is invalid or expired, the system displays an error.

8.3.2 Payment Declined

If the payment fails, the system prompts the user to retry or choose another method.

8.3.3 Session Timeout

If the user takes too long, the session expires and the basket may be reset.

8.4 Special Requirements

- PCI-compliant payment processing
- Secure promo code validation
- Real-time transaction feedback

8.5 Pre-Conditions

- Basket contains at least one item
- User is authenticated

8.6 Post-Conditions

- Payment is completed and confirmed
- Basket is cleared
- Receipt is generated

8.7 Extension Points

- Integration with third-party payment gateways
- Email or SMS confirmation

9. Product Management

9.1 Brief Description

This use case allows an admin to manage the product catalog, including adding, editing, deleting, and categorizing tickets and merchandise.

9.2 Flow of Events

9.2.1 Basic Flow

This use case starts when an admin accesses the product management dashboard.

1. The system displays the list of products.
2. The admin selects an action: Add, Edit, Delete, or Categorize.
3. The system executes the corresponding subflow.

9.2.1.1 Add Product

1. The admin clicks “Add Product.”
2. The system prompts for product details (name, description, price, stock, category).
3. The system validates and saves the product.

9.2.1.2 Edit Product

1. The admin selects a product.
2. The system displays current details.
3. The admin updates fields.
4. The system saves changes.

9.2.1.3 Delete Product

1. The admin selects a product to delete.
2. The system prompts for confirmation.
3. Upon confirmation, the product is removed.

9.2.1.4 Categorize Product

1. The admin selects a product.
2. The admin assigns one or more categories.
3. The system saves the categorization.

9.3 Alternative Flows

9.3.1 Product Not Found

If the selected product does not exist, the system displays an error.

9.3.2 Invalid Input

If required fields are missing or invalid, the system prompts for correction.

9.3.3 Product Linked to Active Orders

If the product is linked to active orders, deletion may be restricted.

9.4 Special Requirements

- Admin-only access
- Audit logging for changes
- Stock validation

9.5 Pre-Conditions

- Admin is authenticated
- Product catalog is accessible

9.6 Post-Conditions

- Product is added, updated, deleted, or categorized
- Catalog reflects latest changes

9.7 Extension Points

- Scheduled publishing
- Bulk import/export
- Analytics dashboard

10. Maintain User Management

10.1 Brief Description

This use case allows users and administrators to manage user accounts within the RYMS app. It includes actions such as creating accounts, logging in/out, assigning roles, editing profiles, resetting passwords, and activating/deactivating accounts.

10.2 Flow of Events

10.2.1 Basic Flow

This use case starts when a user or administrator wishes to perform one of the following actions:

- Create Account
- Login
- Logout
- Assign Role
- View/Edit Profile
- Change Password
- Reset/Forgot Password
- Deactivate/Reactivate Account

Once the actor selects the desired action, one of the subflows is executed:

- If the actor selects **Create Account**, the *Create Account* subflow is executed.
- If the actor selects **Login**, the *Login* subflow is executed.
- If the actor selects **Logout**, the *Logout* subflow is executed.
- If the admin selects **Assign Role**, the *Assign Role* subflow is executed.
- If the actor selects **View/Edit Profile**, the *View/Edit Profile* subflow is executed.
- If the actor selects **Change Password**, the *Change Password* subflow is executed.
- If the actor selects **Reset/Forgot Password**, the *Password Reset* subflow is executed.
- If the actor selects **Deactivate/Reactivate Account**, the *Account Status Management* subflow is executed.

10.2.1.1 Create Account

- The system prompts the user to enter required information (e.g., name, email, password).

- The user submits the form.
- The system validates the input and creates the account.
- The system confirms account creation and may send a verification email.

10.2.1.2 Login

- The system prompts the user to enter their username and password.
- The user enters credentials.
- The system validates the credentials.
- If valid, the user is logged in and redirected to the dashboard.

10.2.1.4 Assign Role

- The admin selects a user account.
- The admin chooses a role (e.g., Admin, Employee).
- The system updates the user's role and confirms the change.

10.2.1.5 View/Edit Profile

- The user accesses their profile settings.
- The system displays current profile information.
- The user updates fields (e.g., name, contact info).
- The system validates and saves the changes.

10.2.1.7 Password Reset / Forgot Password

- The user selects "Forgot Password."
- The system prompts for registered email.
- The user enters email.
- The system sends a reset link.
- The user clicks the link and sets a new password.
- The system confirms the reset.

10.2.1.8 Deactivate/Reactivate Account

- The user accesses account settings.
- The user selects "Deactivate Account."
- The system prompts for confirmation.
- Upon confirmation, the account is deactivated.
- To reactivate, the user logs in and confirms reactivation.
- The system reactivates the account.

10.3 Alternative Flows

10.3.1 Invalid Credentials

If login credentials are incorrect, the system displays an error and prompts for retry.

10.3.2 Duplicate Account

If the email is already registered during account creation, the system displays an error.

10.3.3 Unauthorized Role Assignment

If a non-admin attempts to assign roles, the system denies access.

10.3.4 Invalid Password Reset Request

If the email is not registered, the system displays an error.

10.3.6 Deactivation Cancelled

If the user cancels the deactivation process, the account remains active.

10.4 Special Requirements

- Secure password storage and validation
- Role-based access control
- Email verification for account creation and password reset
- Audit logging for role changes and account status updates

10.5 Pre-Conditions

- User must have access to the RYMS app
- Admin must be authenticated for role assignment and account management

10.6 Post-Conditions

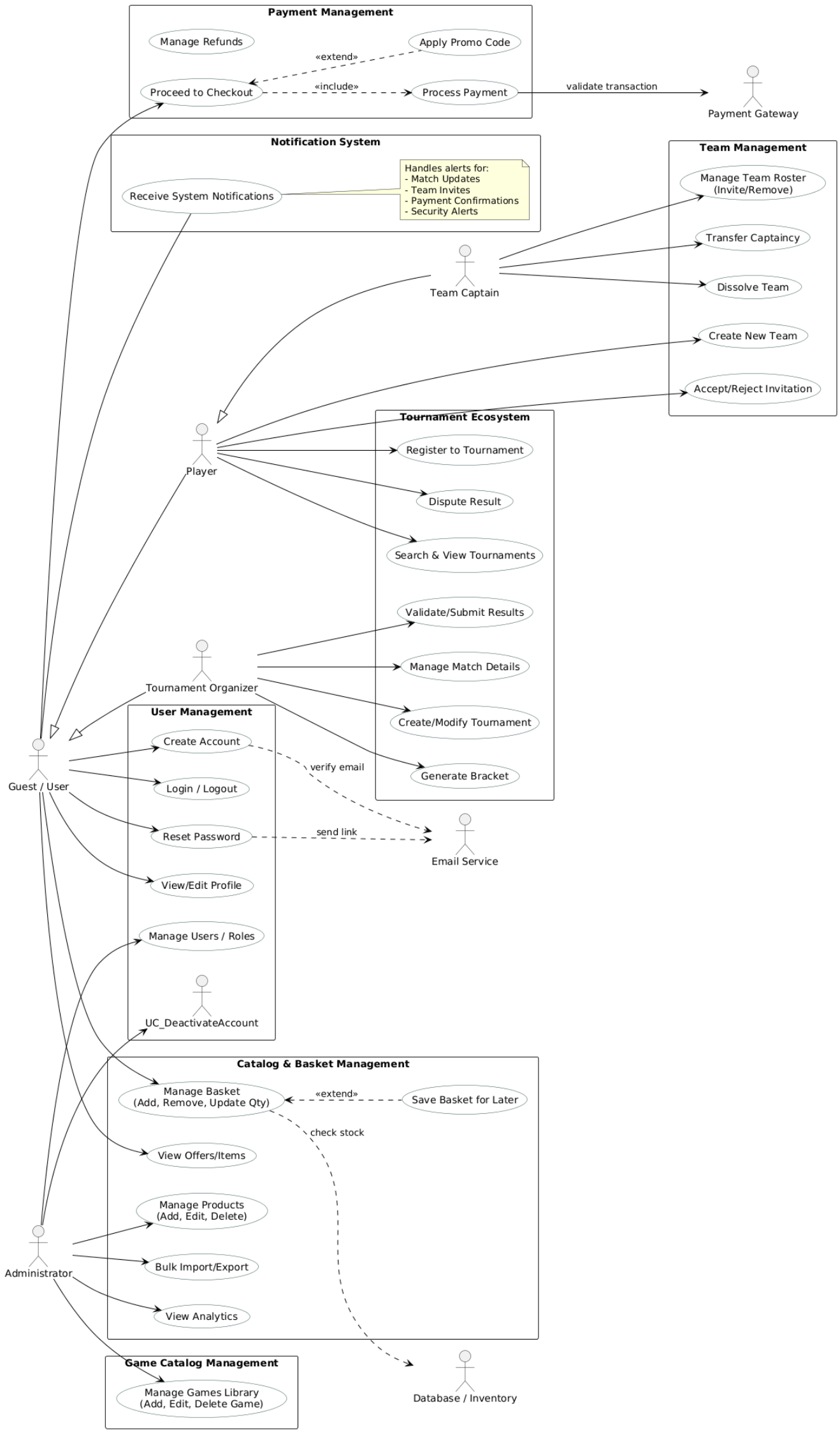
- User account is created, updated, or deleted
- User is logged in or out
- Password is changed or reset
- Role is assigned
- Account status is updated

10.7 Extension Points

- Multi-factor authentication

RYMS eSport – User Management

- Integration with external identity providers
- Scheduled account deactivation/reactivation
- Profile picture upload



Liste des jeux

Administrator View

Liste des jeux

CS:GO

EditDelete

VALORANT

EditDelete

League of Legends

EditDelete

Rocket League

EditDelete

Street Fighter

EditDelete

Overwatch

EditDelete

Dota 2

EditDelete

Apex Legends

EditDelete

Ajouter Un jeu

Nom du Jeu

Editeur du Jeu

Date de Sortie

Bannière du Jeu

Add

Are you sure you want to delete this game ?

ConfirmCancel

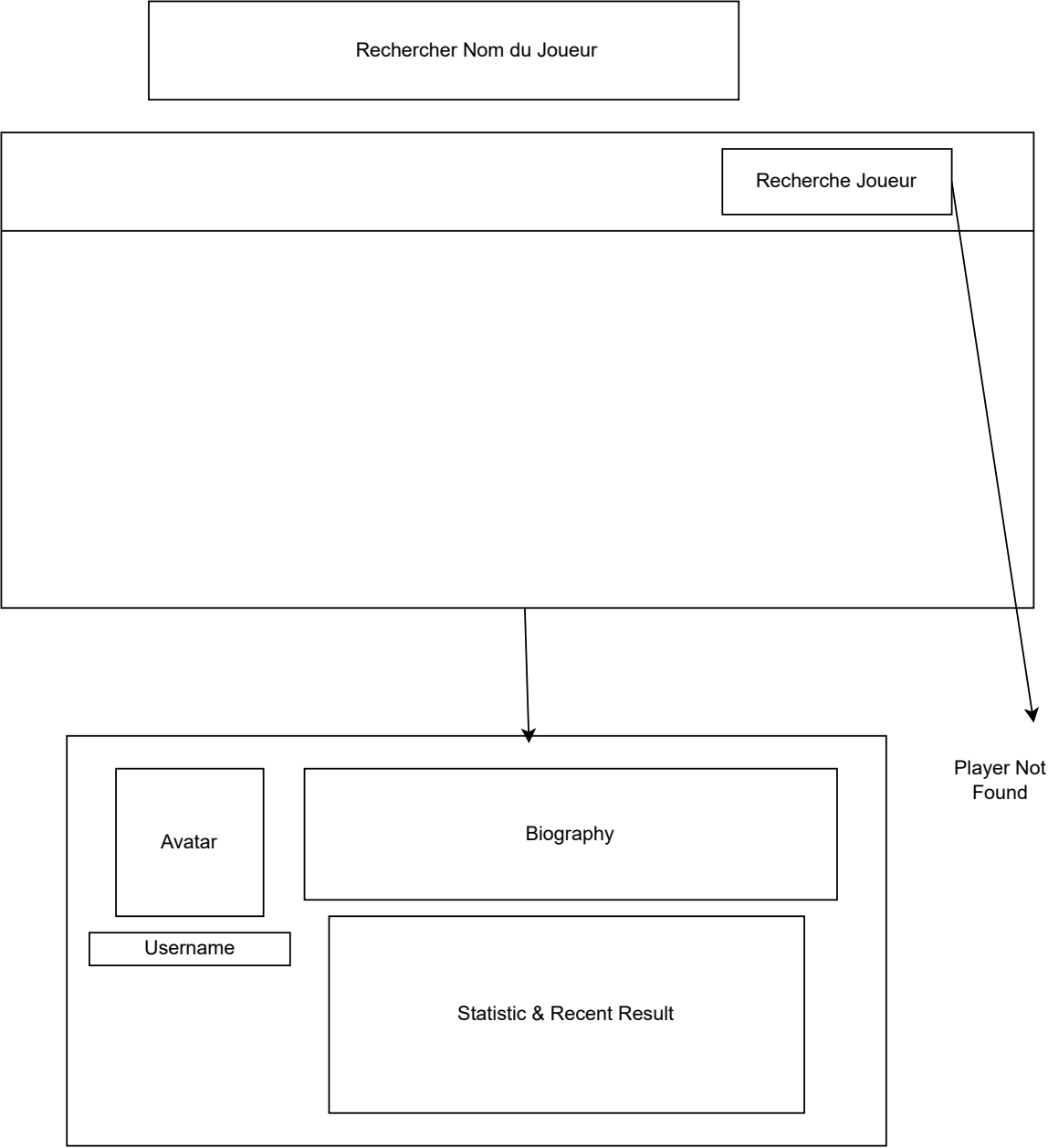
Nom du Jeu

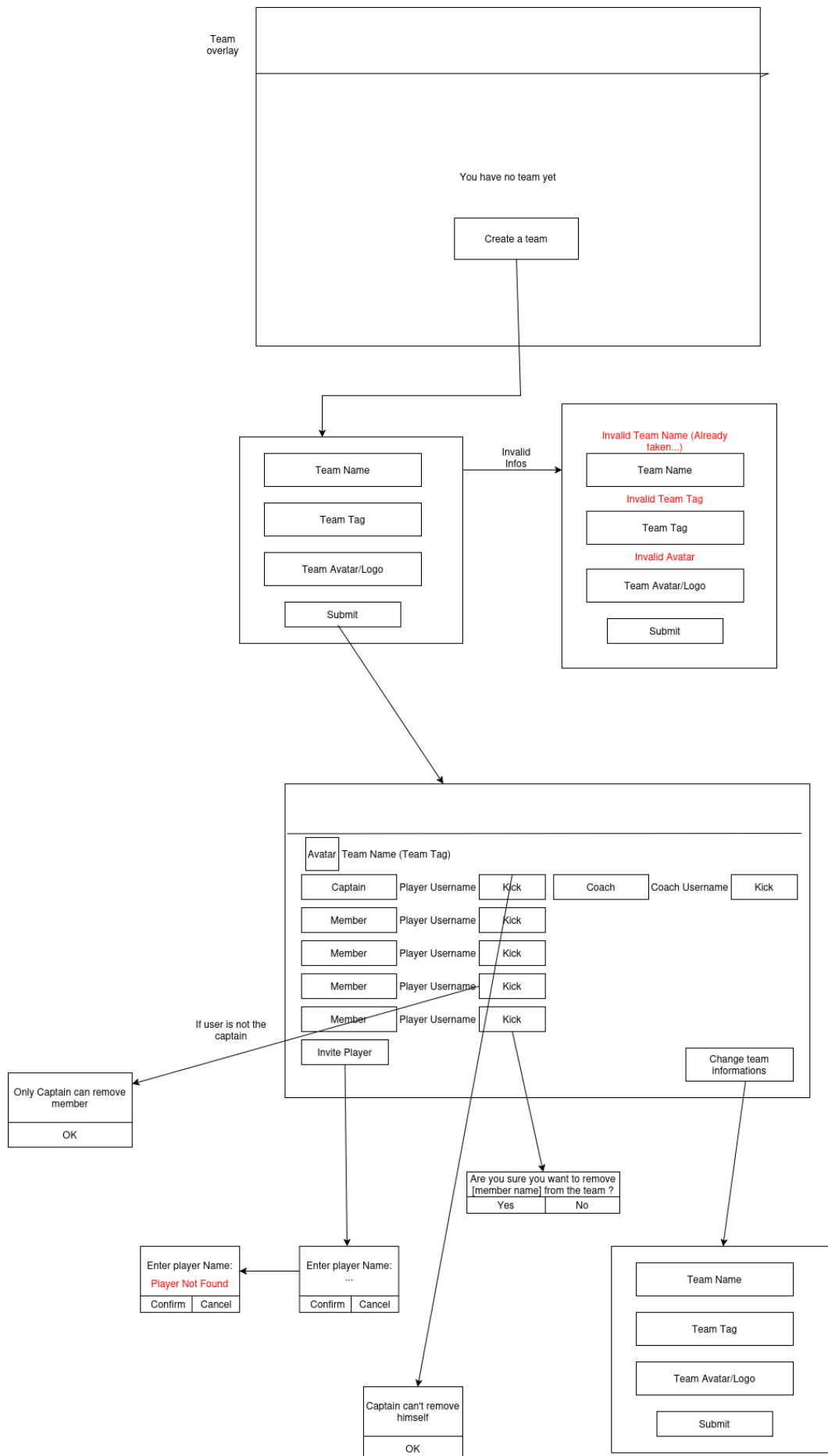
Editeur du Jeu

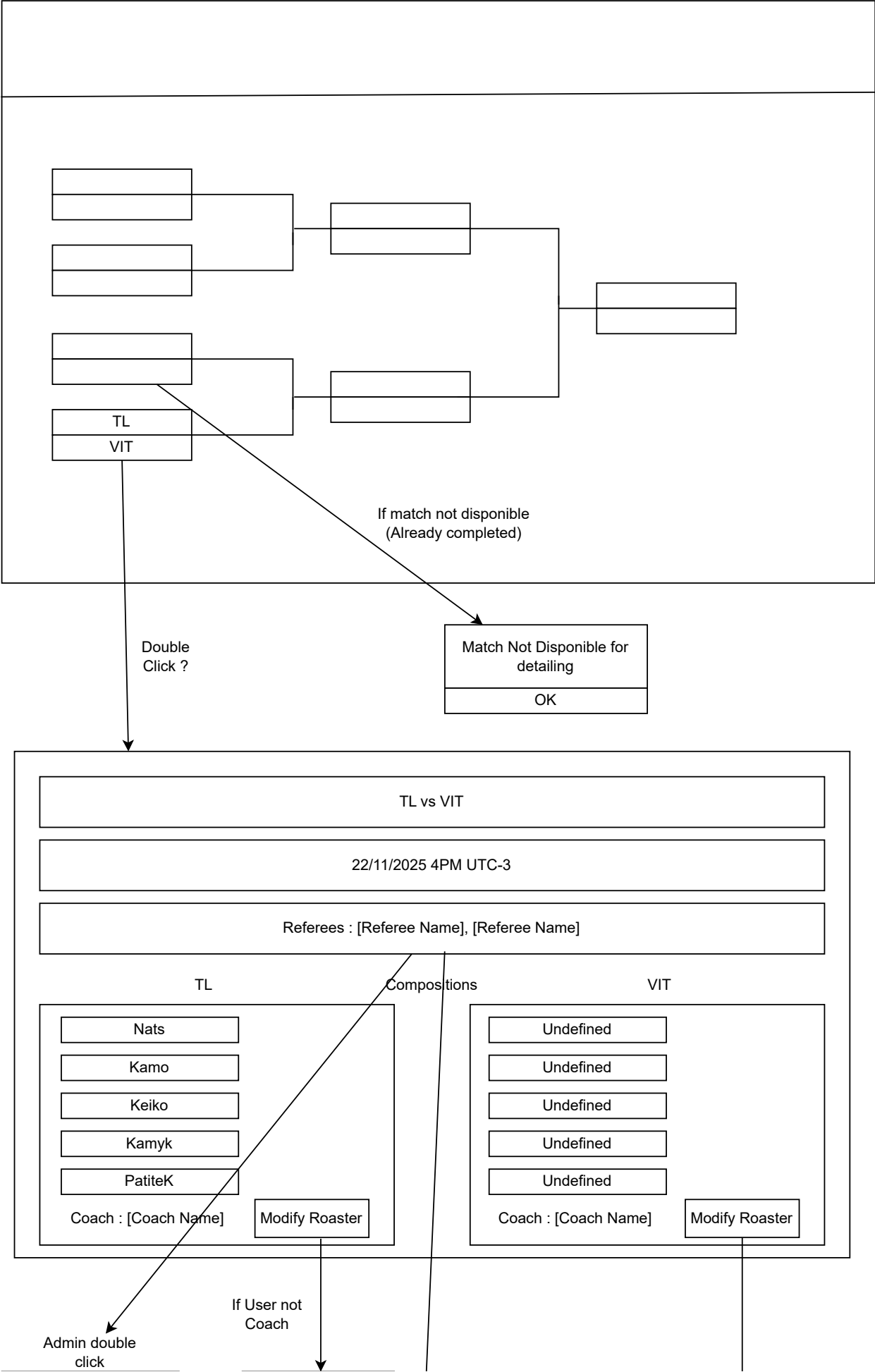
Date de Sortie

Bannière du Jeu

Save Changes







Referee 1

Referee 2

Save

Only the coach can
modify the roster

OK

Not admin

Only Admin can modify
this field

OK

Can Modify
fields

Undefined

Undefined

Undefined

Undefined

Undefined

Save

Player not in the team

Sayonara

Chronicle

Player Not Available

Derke

Must Not Be Empty

Undefined

Must Not Be Empty

Undefined

Save

photo

product

Quantity

delete

Empty Basket

Total

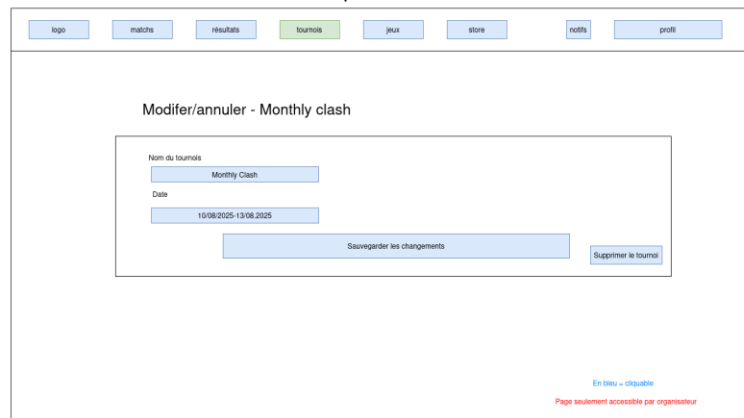
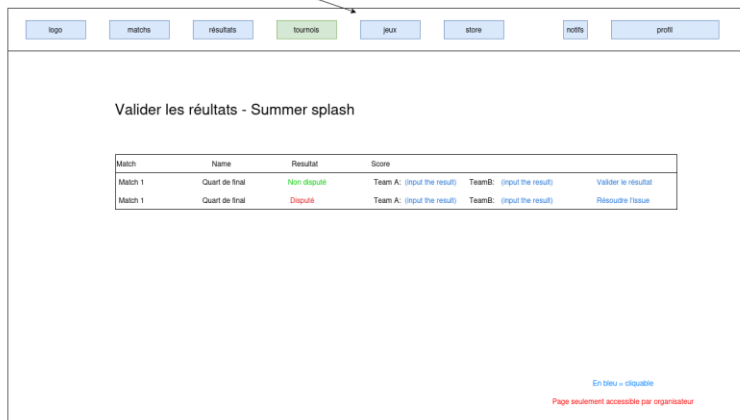
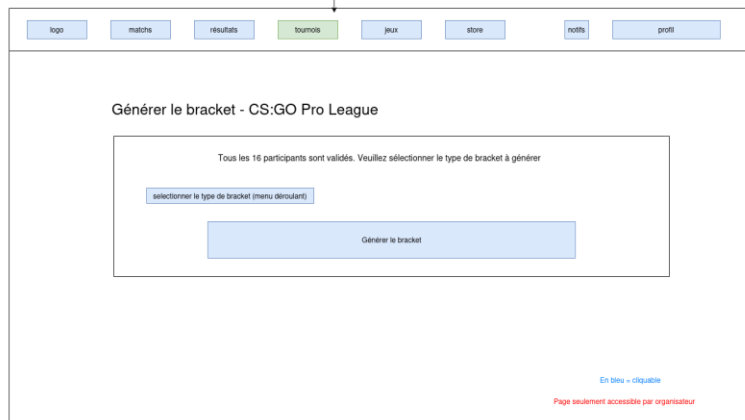
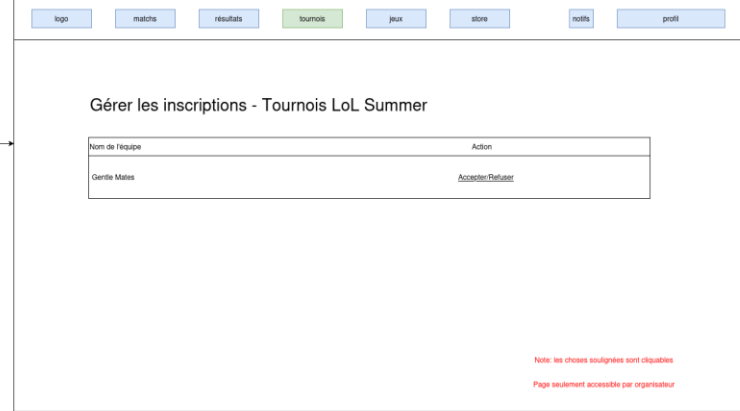
Pay

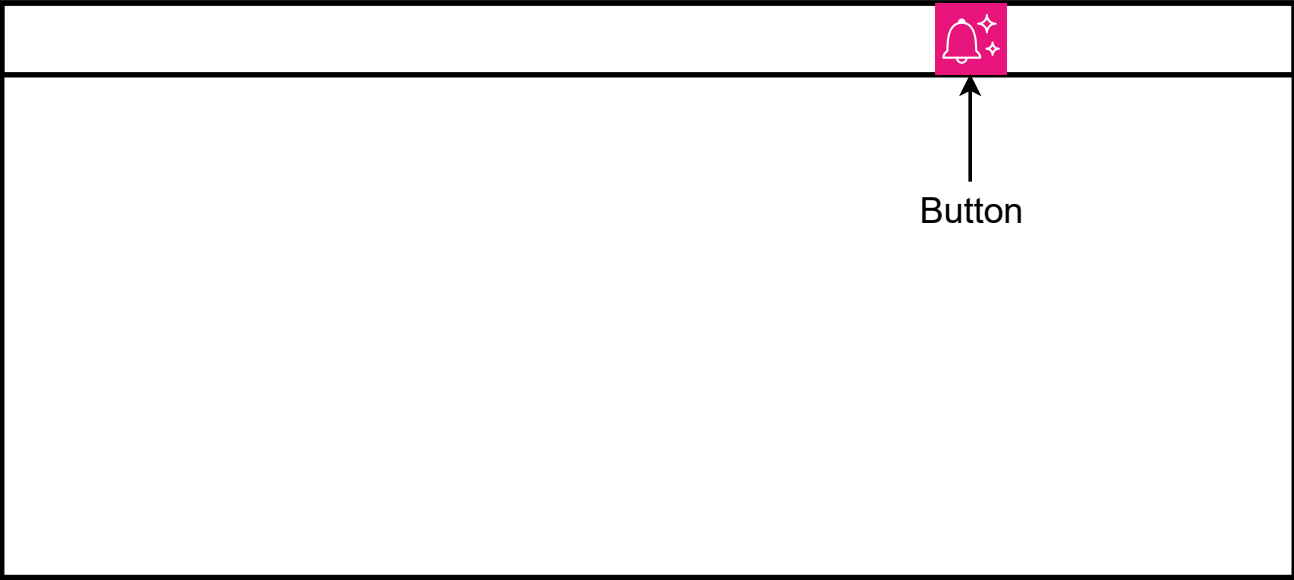
photo

product

price

add to cart





	Notifications
	Heure : Message 
	Heure : Message 
	Heure : Message 

Match

Results

Tournament

Games

Store



profile

Summary

Promo code

Apply

Card info

Pay



Match

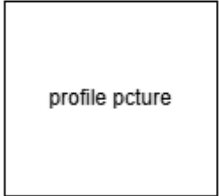
Results

Tournament

Games

Store

Notif



- List
- info

commands

favorites

security

Products

Team

users

Product			add		
Name	In stock	Category			
P1	10	Object	Edit	Visible ?	Delete
Px ...	...	...	Edit	Visible ?	Delete

